

**Aim:** Implement Artificial Neural Network training process in Python by using Forward Propagation, Back Propagation. **Objective:** To learn about Forward Propagation and Backpropagation in ANN Theory: **Forward propagation:** Forward propagation is where input data is fed through a network, in a forward direction, to generate an output. The data is accepted by hidden layers and processed, as per the activation function, and moves to the successive layer. The forward flow of data is designed to avoid data moving in a circular motion, which does not generate an output. **Back propagation:** Backward Propagation is the process of moving from right (output layer) to left (input layer). Forward propagation is the way data moves from left (input layer) to right (output layer) in the neural network. A neural network can be understood by a collection of connected input/output nodes. The accuracy of a node is expressed as a loss function or error rate. Backpropagation calculates the slope of a loss function of other weights in the neural network. **Algorithms:** **Inputs:** X: a training example input vector w: the weight matrix for the network b: the bias vector for the network **Outputs:** a\_L: the output of the final layer of the network **Steps:** Set a\_0 = X, the input vector for the network. For each layer | in the network, compute the weighted input z\_l for that layer:  $Z = w * a_{-1} + b$  Apply the activation function g to the weighted input for each layer to compute the activation a\_l:  $a = g(Z)$  Repeat steps 2 and 3 for all layers in the network, up to the final layer L. Return a\_L as the output of the network for input X. **Applications:** Subject Code:317531 Subject Name: Software Laboratory II ( SL II) SEM-II 26 Department of Artificial Intelligence and Data Science Engineering, ADYPSOE Applications of Feedforward Neural Networks and back propagation: Pattern recognition. Classification tasks. Regression analysis. Image recognition. Computer vision Speech recognition Natural language processing Robotics and even medical diagnosis

### Code

```
import numpy as np

# Activation function (Sigmoid) and its derivative
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x) # Derivative of sigmoid

# Training data (XOR problem)
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]]) # Input features
y = np.array([[0], [1], [1], [0]]) # Target labels

# Initialize weights and biases randomly
np.random.seed(1)

input_neurons, hidden_neurons, output_neurons = 2, 4, 1
```

```

weights_input_hidden = np.random.rand(input_neurons, hidden_neurons)
weights_hidden_output = np.random.rand(hidden_neurons, output_neurons)
bias_hidden = np.random.rand(1, hidden_neurons)
bias_output = np.random.rand(1, output_neurons)

# Training process

epochs = 10000
learning_rate = 0.1

for epoch in range(epochs):
    # **Forward Propagation**

    hidden_input = np.dot(X, weights_input_hidden) + bias_hidden
    hidden_output = sigmoid(hidden_input) # Activation at hidden layer

    final_input = np.dot(hidden_output, weights_hidden_output) + bias_output
    final_output = sigmoid(final_input) # Activation at output layer

    # **Backpropagation**

    error = y - final_output # Compute error
    d_output = error * sigmoid_derivative(final_output) # Output layer gradient

    error_hidden = d_output.dot(weights_hidden_output.T)
    d_hidden = error_hidden * sigmoid_derivative(hidden_output) # Hidden layer gradient

    # **Update Weights and Biases**

    weights_hidden_output += hidden_output.T.dot(d_output) * learning_rate
    weights_input_hidden += X.T.dot(d_hidden) * learning_rate
    bias_output += np.sum(d_output, axis=0, keepdims=True) * learning_rate
    bias_hidden += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate

    # Print loss every 1000 epochs

    if epoch % 1000 == 0:
        loss = np.mean(np.abs(error))

```

```
print(f"Epoch {epoch}, Loss: {loss:.4f}")
```

```
# **Testing**
```

```
print("\nFinal Predictions:")
```

```
for i in range(len(X)):
```

```
    hidden_layer = sigmoid(np.dot(X[i], weights_input_hidden) + bias_hidden)
```

```
    output = sigmoid(np.dot(hidden_layer, weights_hidden_output) + bias_output)
```

```
    print(f"Input: {X[i]} => Predicted: {output[0][0]:.4f}")
```